

JavaScript Type Coercion

[Back to JS page](#)

Table of Contents

- [JavaScript Type Coercion](#)
 - [Type Coercion](#)
 - [Example 1](#)
 - [String Coercion \(+\)](#)
 - [Example 2](#)
 - [Numeric Coercion](#)
 - [Example 3](#)
 - [Boolean Coercion](#)
 - [Example 4](#)
 - [Loose Equality \(==\)](#)
 - [Example 5](#)
 - [Best Practices to Avoid Bugs](#)
 - [Example 6](#)
 - [Document](#)
 - [Reference](#)

Type Coercion

Type coercion is the **automatic conversion** of values from one data type to another.

Type coercion happens when you perform an operation on different data types, and the JavaScript engine tries to make them "fit" together.

JavaScript coerces types differently per operator, so type coercion can hide bugs. The program continues according to the coercion rules, but these may differ from what the programmer wanted to achieve.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Type Coercion</h4>
<p id="demo"></p>

<script>
let result1 = ('5' + '2'); // = 52
let result2 = ('5' - '2'); // = 3

document.getElementById("demo").innerHTML =
  "('5' + '2') = " + result1 + "<br>" +
  "('5' - '2') = " + result2;
</script>

</body>
</html>
```

Coercion is **implicit** (handled automatically by the engine), while **type conversion** is explicit (you manually use functions like `Number()` or `String()`).

Because JavaScript is **weakly typed**, it doesn't throw errors when types don't match; it just tries to coerce them.

Example 1

Result [View Example](#)

String Coercion (+)

If any part of a `+` operation is a string, JavaScript converts everything to strings and concatenates them.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>String Coercion (+)</h4>
<p id="demo"></p>

<script>
let x = "5" + 2 // x = "52"

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Numeric Coercion

Other arithmetic operators (`-` , `*` , `/` , `%`) and the unary plus (`+x`) force values into numbers.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Numeric Coercion</h4>
<p id="demo"></p>

<script>
let x = "5" - 2 // x = 3

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Boolean Coercion

Values are coerced to booleans in logical contexts like `if` statements or using the double-NOT operator (`!!`).

- **Falsy values:** `0`, `""`, `null`, `undefined`, `NaN`, and `false`
- **Truthy values:** Everything else (including empty objects `{}` and arrays `[]`)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Boolean Coercion</h4>
<p id="demo"></p>

<script>
let text = "";

text += "Boolean(0) = " + Boolean(0) + "<br>";
text += "Boolean(\"\") = " + Boolean("") + "<br>";
text += "Boolean(null) = " + Boolean(null) + "<br>";
text += "Boolean(undefined) = " + Boolean(undefined) + "<br>";
text += "Boolean(NaN) = " + Boolean(NaN) + "<br>";
text += "Boolean(false) = " + Boolean(false) + "<br>";
text += "Boolean({}) = " + Boolean({}) + "<br>";
text += "Boolean([]) = " + Boolean([]) + "<br>";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Loose Equality (==)

This operator performs coercion to find a "common type" before comparing.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Loose Equality (==)</h4>
<p id="demo"></p>

<script>
let x = (5 == "5") // x = true

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Best Practices to Avoid Bugs

- **Use === (Strict Equality):** This checks both value and type without coercion, preventing weird results.
- **Be Explicit:** Instead of relying on automatic behavior, use MDN's recommended conversion methods like `Number()` or `String()` to make your intent clear.
- **Watch for NaN:** If a string cannot be converted to a valid number, like `"abc" - 1`, the result is `NaN` (Not-a-Number).

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Type Coercion</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Best Practices to Avoid Bugs</h4>
<p id="demo"></p>

<script>
let text = "";

// Use === (Strict Equality)
text += "5 == \"5\" : " + (5 == "5") + "<br>";
text += "5 === \"5\" : " + (5 === "5") + "<br><br>";

// Be Explicit
text += "Number(\"5\") + 2 : " + (Number("5") + 2) + "<br><br>";

// Watch for NaN
let result = "abc" - 1;
text += "\"abc\" - 1 : " + result;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Type Coercion](#)